

REALIZZAZIONE MACCHINA ASTRATTA

Strada Interpretativa
⇓

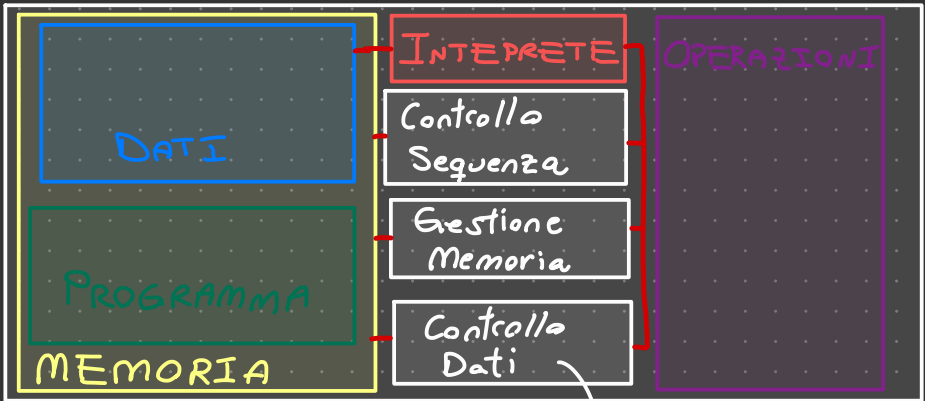
Esecuzione Istantanea
e viene Eseguita la **Traduzione**
Istruzione per Istruzione
in L_0

Strada Compilativa
⇓

Viene Tradotto Completamente
 L in L_0 (Linguaggio Eseguitabile)
della Macchina Sottostante

Interprete da L in L_0 : $\approx I^{L_0 L} (P^L, \text{Input}) = \text{Output}$
 $\approx I^{L_0 L}: \text{Prog}^L \times D \rightarrow D$

La Macchina Astratta M_{L_0} :



allocazione di DATI

INTERPRETE DALL'INTERNO:

Simulazione SW di Cosa Succede Sulla Macchina:

BEGIN

go := TRUE

WHILE go DO: AT

FETCH(OPCODE, OPINFO) ^{AT} PROG. COUNTER // Estraggo dalla Memoria

DECODE(OPCODE, OPINFO)

IF OPCODE needs AGRS // Devo Estrarli dalla MEMORIA

THEN fetch(ARGS)

CASE opcode off:

op 1: EXECUTE(OP1, args)

CONTROLLO DATI

op N: EXECUTE(OPN, args)

GESTIONE MEMORIA

HLT: go := false

CONTROLLO SEQUENZA

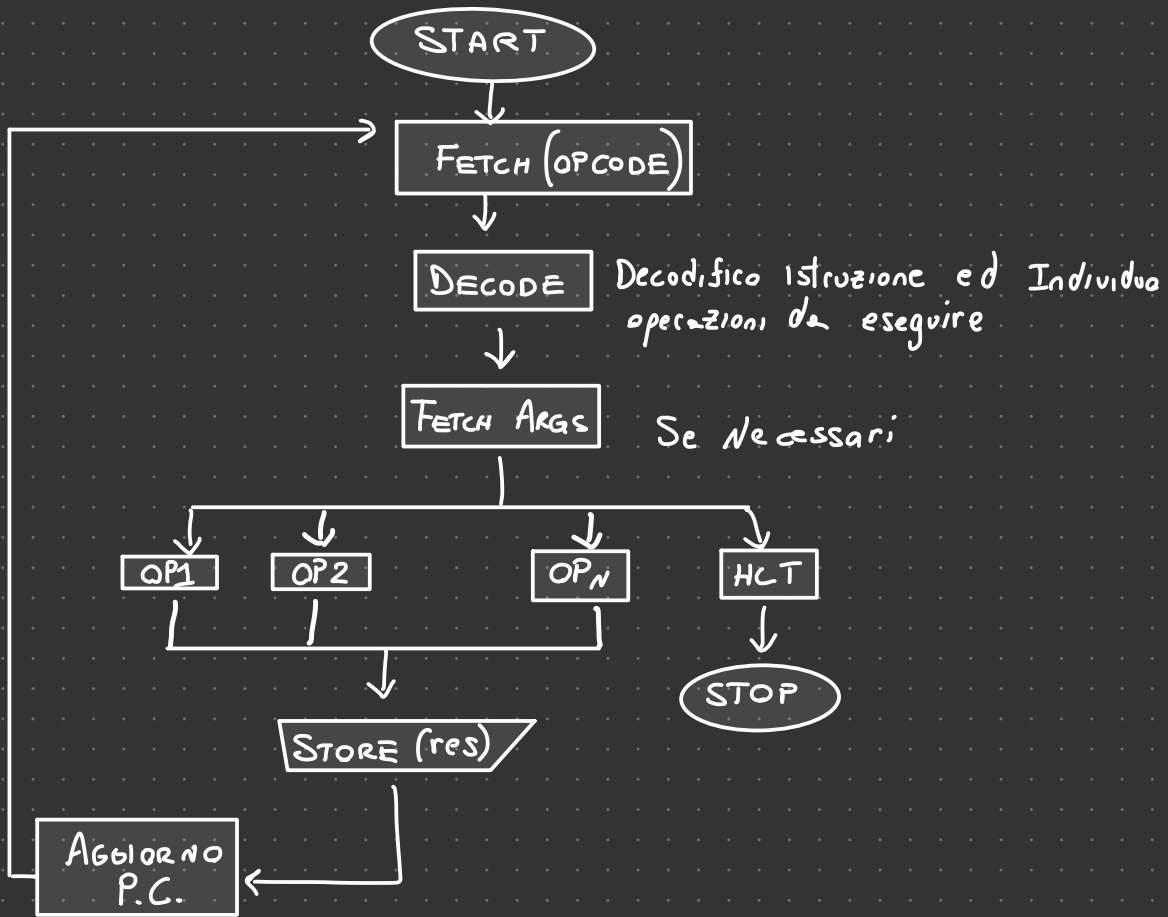
IF opcode Salva Results:

STORE(results)

PC. = P.C. + (size(opcode))

END

IN FLOW CHART:



SOLUZIONE COMPILATIVA:

Macchina Astratta che Realizziamo fa lavoro Solo Sintattico $\Rightarrow$ DEVE TRADURRE (e BASTA)

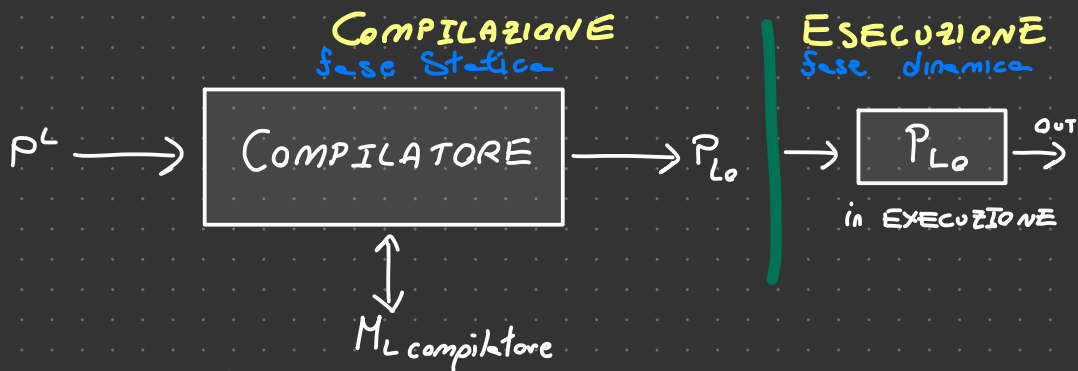
COMPILATORE $\Rightarrow$ da L a L_o $\overset{\text{SORGENTE}}{\leftarrow}$ $\overset{\text{TARGET}}{\rightarrow}$ È un PROGRAMMA:

$C^{L L_o}: \text{Prog}^L \rightarrow \text{Prog}^{L_o}$ (non guarda i dati Lui)

$$C^{L L_o}(P^L) = P^{L_o} \text{ t.c. } \forall d \in D \quad P^L(d) = P^{L_o}(d)$$

Eguivalenza SEMANTICA dei linguaggi L ed L_o (ovviamente $\neq$ SINTATTICAMENTE)

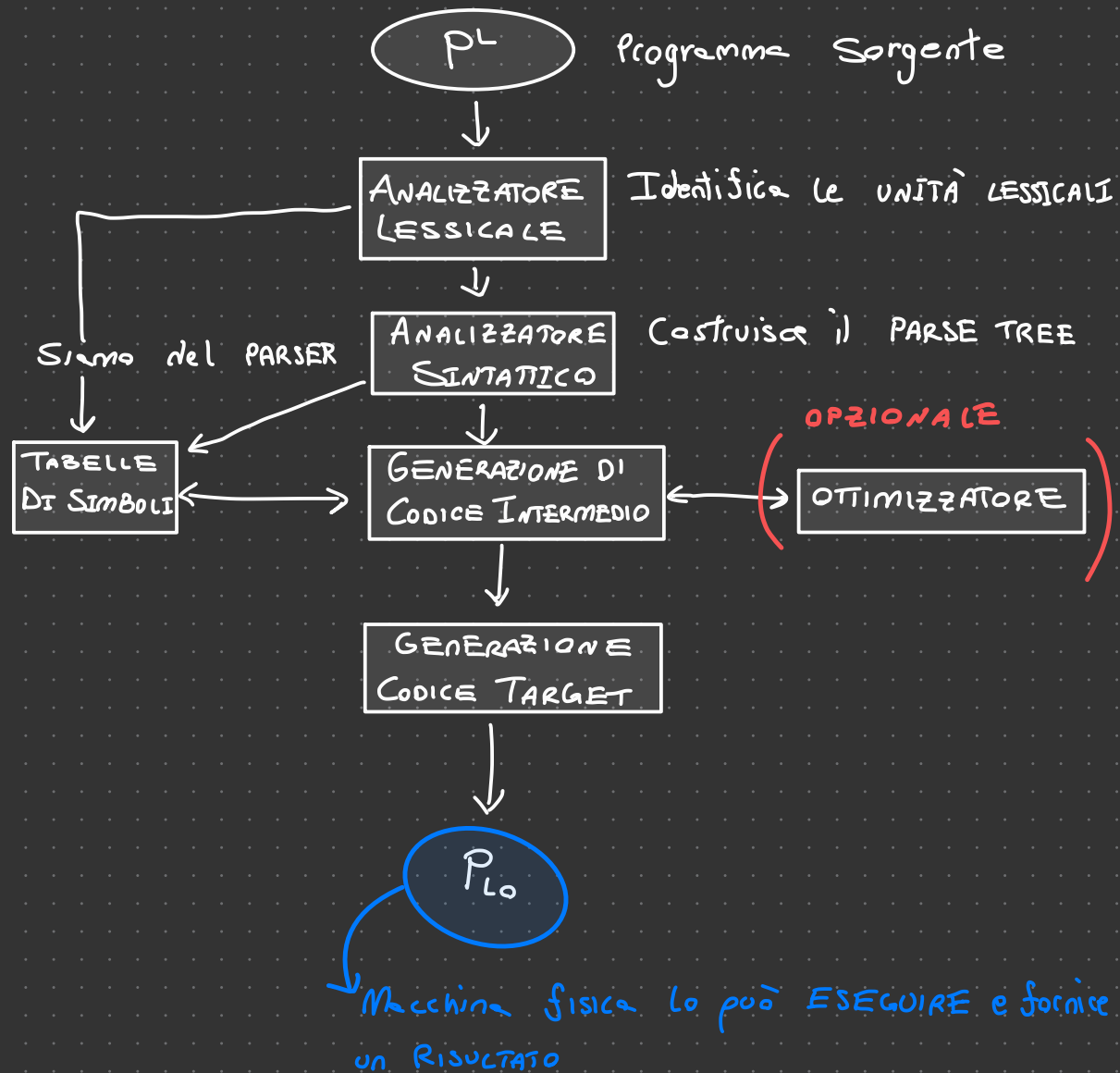
$\Rightarrow$ $\forall$ INPUT comportamento È uguale



Macchina Abs. del linguaggio per Scrivere il Compilatore

Ci Sono ANALISI Statiche per OTIMIZZARE il Codice in fase di TRADUZIONE per Avere poi un ESECUZIONE ancora più Veloce

STRUTTURA:



L da Implementare

Lo linguaggio Implementato



$L_I \rightarrow$ linguaggio Intermedio

$M_L \xrightarrow{\text{COMPILAZIONE}} M_I \equiv M_O$

$M_I = M_L \xrightarrow{\text{INTERPRETAZIONE}} M_O$

PROIEZIONI DI FITAMURA:

Specializzatore $\Rightarrow SPEC_L: (prog^L \times \text{Dato}) \rightarrow prog^L$

$SPEC_L: (p^L, d) = P_d$ $\rightarrow$ porzione INPUT

$\forall y. P_L(y) = P_d^L(y)$ $\rightarrow$ restante PORZIONE

Useto ad Esempio per Teorema smn

Specializzare un Interprete su un programma da Eseguire
restituisce un COMPILATORE

DESCRIVERE UN LINGUAGGIO DI PROGRAMMAZIONE:

- /// **SINTASSI** $\Rightarrow$ Regole di formazione/costruzioni frasi ben formate
descrive **RELAZIONI** tra **Simboli/Segni**
- /// **SEMANTICA** $\Rightarrow$ Attribuzione del **Significato ai Simboli**, data
una frase che **RISPETTA** le **REGOLE SINTATTICHE**
- /// **PRAGMATICA** $\Rightarrow$ Sono **REGOLE di BUON UTILIZZO** (Ingegneria
del SOFTWARE) che **NON** sono **OBBLIGATORIE** $\Rightarrow$ come
programmare **BENE**
- /// **IMPLEMENTAZIONE** $\Rightarrow$ Come si Eseguono frasi ben formate
che hanno Significato

SINTASSI:

Consiste nel descrivere le regole di formazione del linguaggio

- /// **PAROLA** $\Rightarrow$ Stringa di caratteri
- /// **FRASE** $\Rightarrow$ Sequenza di parole ben formate (**RISPETTA REGOLE SINTASSI**)
- /// **LINGUAGGIO** $\Rightarrow$ Insieme delle frasi
- /// **LESSEMA** $\Rightarrow$ Identifica una parola con significato specifico
(**SIMBOLI TERMINALI DELLA GRAMMATICA**).
È **UNITÀ SINTATTICA** del linguaggio
- /// **TOKEN** $\Rightarrow$ È una **Categoria** sintattica (**SIMBOLI NON TERM.**)

NELLE GRAMMATICHE)

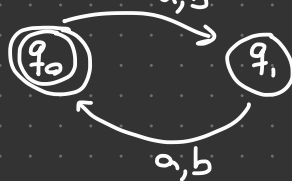
RICONOSCI TORI

automi lo sono

GENERATORI

Grammatiche

$L = \{s \mid |s| \text{ è Pari} \}$ posso Scrivere sia automa Riconoscitore
con $\Sigma = \{a, b\}$



O una Grammatica Generatrice:

$G: S \rightarrow \epsilon \mid aaS \mid abS \mid bbS \mid baS$

Descrivo la Grammatica

ALFABETO $\Rightarrow$ Simboli

LESSICO $\Rightarrow$ Sequenze di Simboli che Costruiscono Parole
ovvero i **SIMBOLI TERMINALI**

REGOLE SINTATTICHE $\Rightarrow$ Regole di derivazione della Grammatica
che Compongono Tra loro Parole.

CATEGORIE SINTATTICHE $\Rightarrow$ **SIMBOLI NON TERMINALI** della
Grammatica

Si usano **Grammatiche C.F.** anche se NON catturano TUTTO come

una chiamata a Procedura.

Catturano UNA PARTE delle Regole ma NON Tutto.

Si usano le Grammatiche C.F. per gli Algoritmi Efficienti
che hanno  anche dette BNF

Apertura e chiusura Parentesi è la causa che ci ha fatto
Abbandonare gli AUTOMI altrimenti Sarebbero stati ancora
più EFFICIENTI

GRAMMATICHE:

È una Quintupla $G = \langle V, T, P, S \rangle$

V insieme finito di Simboli non Terminali (CATEGORIE SINTATT.)

quindi espressioni, Comandi, Procedure, dichiarazioni,

Sono gli Elementi della frase (TOKEN)

T Sono i Simboli TERMINALI che hanno già un SIGNIFICATO visto che hanno definizione nel nostro Vocabolario (LESSEMI) come ad Esempio $\Rightarrow$ WHILE, IF, :=, {, },

P Produzioni $A \rightarrow \alpha$ dove $A \in V$ e $\alpha \in (V \cup T)^*$, Regole di formazione del Programma ad Esempio:

com $\rightarrow$.. WHILE exp DO com ...

S $\in V$ Simbolo Iniziale è la Categoria dei Programmi

AD ESEMPIO:

Exp $\langle \{E\}, \{or, and, not, (,), 0, 1\}, P, E \rangle$

$\rightarrow E \rightarrow 0 \mid 1 \mid (E \text{ or } E) \mid (E \text{ or } E) \mid (NOT \ E)$

definisco i Casi Base

Come Combino le Espressioni $\Rightarrow$ PASSO INDUTTIVO